

Research: Recommender System

Primitive Atoms

Executive Overview and Architectural Paradigm

The evolution of modern recommender systems has increasingly shifted from monolithic, black-box architectures toward modular, decoupled pipelines. In highly scaled environments—such as those managing the 1.8 million unique catalog items of the OTTO webshop or the vast historical transaction repositories of the H&M fashion dataset—the orchestration of candidate generation, feature assembly, and ranking demands rigorous software engineering principles.¹ The prevailing best practice dictates the segregation of stateful orchestration (e.g., database I/O, distributed model training loops) from stateless computation. This architectural division is achieved by designing recommender system building blocks as "pure functions"—mathematical mappings that ingest immutable arrays and parameters to return deterministic transformations without side effects.³

The primary objective of this exhaustive analysis is to define best-in-class, pure-function implementations for core recommender system primitives, specifically targeting the sciona-atoms-dl repository. These primitives span three primary operational phases: searching (for candidate generation logic), optimization (for matrix factorization algorithms), and data_assembly (for intricate feature engineering). By establishing pure boundaries—where inputs are strictly dense or sparse tensors (e.g., NumPy NDArray, SciPy csr_matrix) and scalar configuration values—these functions guarantee determinism, facilitate distributed execution across cluster frameworks, and vastly simplify isolated unit testing.⁵

This report systematically isolates, mathematically formalizes, and technically specifies seven critical primitives derived from leading empirical applications. This includes top-tier algorithmic solutions from the OTTO Multi-Objective Recommender System and the H&M Personalized Fashion Recommendations challenges.⁶ Each function is dissected in terms of its mathematical formulation, algorithmic complexity, hardware acceleration strategies, edge-case handling methodologies, and its integration into a broader ranking ensemble.

To ensure comprehensive alignment with the broader system design, the following table maps the overarching computational stages to the specific conceptual types established for the pure-function atom repository.

Recommender System Stage	Primary Objective	Concept Type	Target Implementation Atom

Co-visitation Matrix Construction	Discover aggregate item-item relationships from sessions	searching	co_occurrence_matrix
Candidate Retrieval	Extract top-K items for a user based on history	searching	cooccurrence_candidates
Collaborative Filtering	Extract latent user/item factors via matrix factorization	optimization	als_user_update / als_item_update
Popularity Baselines	Establish trending items with temporal decay	data_assembly	item_popularity
Contextual Feature Engineering	Extract statistical properties of an active session	data_assembly	session_features
Affinity Feature Engineering	Measure historical relationships between user and item	data_assembly	user_item_features
Ensemble Fusion	Merge multiple retrieval channels without score scaling	searching	reciprocal_rank_fusion

Retrieval and Candidate Generation Primitives

The retrieval stage functions as the widest aperture of the recommendation funnel. It is fundamentally responsible for reducing a catalog of millions of potential items to a highly probable candidate set of approximately 50 to 300 items for a given user session.⁸ The operational priority in this phase is computational efficiency coupled with high theoretical recall, ensuring that the downstream ranking model possesses a rich pool of relevant targets to score.

1. Temporal and Type-Weighted Co-Occurrence Matrices

The co-occurrence (or co-visitation) matrix serves as a foundational retrieval mechanism in session-based recommendation. Empirical evidence from the OTTO competition demonstrates that over 90% of user sessions contain only short-term sequential clicks, making historical co-visitation probabilities highly predictive for immediate future actions.⁹ Rather than treating all interactions equally, state-of-the-art implementations apply both temporal decay and action-type weighting to generate nuanced relationship graphs.⁶

The underlying mathematical formulation of this approach requires assessing the proximity of two items within the same user session. Let a session S be a sequence of events, where each event e is defined by an item i , a timestamp t , and an action type weight w_{type} (for instance, a click might be weighted 1, a cart addition 6, and a final order 3). The co-occurrence weight $C_{i,j}$ between item i and item j across all sessions is formulated as the sum of these weighted interactions:

$$C_{i,j} = \sum_S \sum_{(e_i, e_j) \in S} w_{type}(e_j) \cdot f_{decay}(|t_i - t_j|)$$

In this equation, f_{decay} represents a continuous or step-wise temporal weighting function that penalizes interactions occurring far apart in time.⁶ For example, interactions occurring within two hours of each other might receive a full weight multiplier, while interactions separated by a week decay significantly.

Constructing this matrix over datasets comprising 200 million events is computationally bound by memory limits. Real-world applications frequently leverage GPU-accelerated dataframe libraries, such as RAPIDS cuDF, to execute highly parallelized self-joins on the session ID, followed by rapid aggregations.⁶ However, the pure-function boundary isolated for the scion-atoms-dl repository requires the aggregation of integer sequences into a memory-efficient sparse matrix using native Python ecosystem tools. For item catalogs exceeding 10^6 distinct entities, instantiating dense matrices inevitably triggers catastrophic

out-of-memory exceptions.¹¹

Therefore, the optimal pure-function implementation must ingest user sessions and construct a SciPy Coordinate (COO) sparse matrix. The COO format, which maintains discrete lists of row indices, column indices, and corresponding data values, is highly efficient for the incremental construction of sparse relationships.¹² Once construction is complete, the COO matrix is systematically converted to a Compressed Sparse Row (CSR) format, which allows for highly optimized algorithmic arithmetic operations in downstream candidate retrieval steps.¹¹

A rigorously defined pure function must also handle degenerate inputs gracefully. If an empty list of sessions is provided, the function must deterministically return an empty sparse matrix of the specified geometric shape. The pure boundary strictly prohibits the function from reading raw event logs directly from disk or querying a database; it inherently assumes that upstream state managers have materialized and pre-filtered the session arrays before invocation.

Name: `co_occurrence_matrix`

Description: Constructs a sparse item-item co-occurrence matrix from a list of user sessions, applying optional time-decay weights based on sequential proximity.

Source: Based on NVIDIA RAPIDS OTTO solution architecture and general sparse matrix literature (<https://github.com/rapidsai/deeplearning/tree/main/OTTO-recommender-system>).

License: Apache-2.0

Concept type: searching

Signature: `(sessions: list[list[int]], n_items: int, time_weights: NDArray | None) -> scipy.sparse.csr_matrix`

Pure function boundary: Ingests jagged lists of item IDs and an optional 1D array of temporal decay weights; outputs a square CSR sparse matrix. No database connections, global state mutations, or disk reads occur.

Contracts:

- require: `n_items > 0`
- require: `max(item_id) < n_items` for all `item_id` in sessions
- require: `time_weights` is `None` OR `len(time_weights) >= max(len(session))`
- ensure: output matrix shape is strictly `(n_items, n_items)`
- ensure: output matrix diagonal is heavily penalized or explicitly zeroed to prevent self-recommendation feedback loops.

Witness: Pass two sessions `[, ]` with uniform weights; assert the resulting matrix reflects a cumulative co-occurrence score of 2.

Dependencies: `numpy`, `scipy.sparse`

CDG stages covered: `otto/co_visitation`, `h_and_m/item_item_cf`

2. Candidate Generation from Co-Occurrence

Once the global co-occurrence matrix is successfully materialized, the subsequent computational requirement is to dynamically retrieve the top K related items for a given user based entirely on their recent, localized interaction history. In the H&M challenge, utilizing localized historical matrices efficiently narrowed massive candidate pools from roughly 105,000 distinct catalog items to just 30 highly relevant prospects per user.⁷

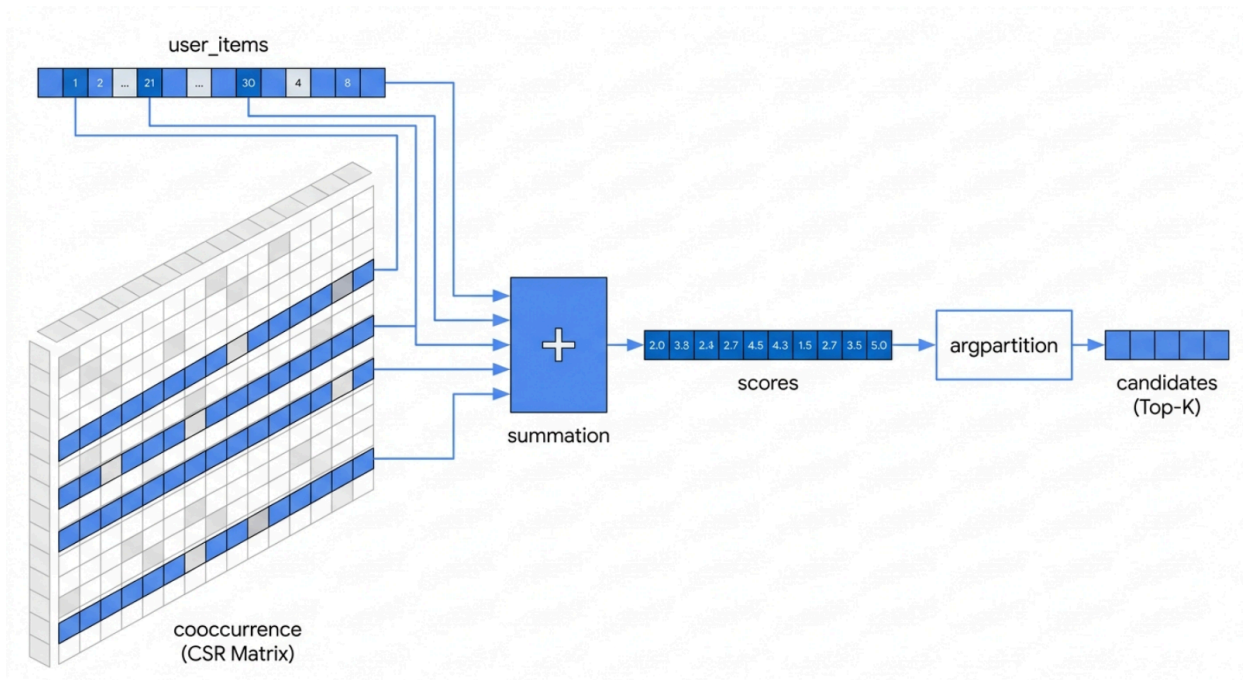
The mathematical implementation of this retrieval step relies on linear algebra. Given a user's recent interaction history represented as a sparse or dense vector $h \in \mathbb{R}^{1 \times |I|}$ (where elements are set to 1 if the user interacted with the item, and 0 otherwise) and the pre-computed item-item co-occurrence matrix $C \in \mathbb{R}^{|I| \times |I|}$, the raw candidate scores are derived via a sparse matrix-vector multiplication:

$$s = h \cdot C$$

The retrieved candidates correspond directly to the specific indices of the highest K values within the resultant score vector s . In practical e-commerce deployments, these candidates are often filtered to exclude items already present in h , specifically when the system's objective is to prioritize novel discovery over repetitive repurchase behaviors.

From a software engineering perspective, a pure implementation must avoid explicitly instantiating massive dense vectors if possible. Instead, it extracts only the specific rows of the CSR matrix corresponding to the user's historical item IDs, sums them along the column axis to accumulate weights, and utilizes `numpy.argpartition` or `numpy.argsort` to efficiently locate the top K elements. The use of `argpartition` is a critical algorithmic optimization; it executes in $O(N)$ time by finding the K -th largest element and partitioning the array around it, completely bypassing the $O(N \log N)$ complexity required for a full array sort.

Data Flow: Pure-Function Candidate Retrieval



The stateless retrieval process extracts relevant rows from the CSR matrix based on user history, aggregates the co-occurrence weights, and performs a partial sort to return the Top-K candidate IDs.

The functional contract guarantees that memory allocation is limited entirely to the output candidate array and an intermediate summation vector, preserving the purity and predictable performance of the system.

Name: `cooccurrence_candidates`

Description: Generates recommendation candidates by aggregating co-occurrence scores corresponding to a user's interaction history, returning the top-K highest scoring items.

Source: Standard collaborative filtering retrieval techniques observed in OTTO Kaggle solutions and H&M retrieval stages.

License: MIT

Concept type: searching

Signature: `(user_items: NDArray, cooccurrence: scipy.sparse.csr_matrix, k: int, filter_seen: bool = True) -> NDArray`

Pure function boundary: Takes a localized array of item IDs and a pre-computed global sparse matrix. Returns a dense 1D array of top-K candidate item IDs. No database access or state

tracking occurs.

Contracts:

- require: $k > 0$
- require: $\text{all}(0 \leq \text{item} < \text{cooccurrence.shape for item in user_items})$
- ensure: $\text{len}(\text{output}) \leq k$
- ensure: if `filter_seen` is `True`, $\text{intersection}(\text{output}, \text{user_items})$ is strictly empty.

Witness: Provide a localized sparse matrix where item 0 co-occurs highly with item 10; pass `user_items=`, ensure item 10 is correctly populated in the top candidate output array.

Dependencies: `numpy`, `scipy.sparse`

CDG stages covered: `otto/candidate_generation`, `h_and_m/retrieval`

Collaborative Filtering and Optimization Components

While co-occurrence matrices handle immediate, short-term sequential relationships exceptionally well, capturing deep, latent user preferences requires the deployment of matrix factorization techniques. The Alternating Least Squares (ALS) algorithm has proven remarkably robust, particularly in e-commerce environments dominated by implicit feedback signals (e.g., clicks, views, hovers) rather than explicit, user-submitted star ratings.¹³

3. Alternating Least Squares (ALS) Closed-Form Update

Matrix factorization fundamentally seeks to approximate a highly sparse user-item interaction

matrix $R \in \mathbb{R}^{U \times I}$ as the dot product of two distinct, lower-rank factor matrices: the user factors $X \in \mathbb{R}^{U \times F}$ and the item factors $Y \in \mathbb{R}^{I \times F}$, where F represents the predetermined latent embedding dimension.¹⁴

Because both the user matrix X and the item matrix Y are simultaneously unknown, the overall optimization problem is mathematically non-convex. The ALS algorithm overcomes this inherent difficulty by fixing one matrix while optimizing the other, effectively reducing the complex problem to a manageable series of convex Ordinary Least Squares (OLS) steps that can be solved directly.¹⁶ For implicit feedback scenarios, a specialized confidence matrix

$C_{ui} = 1 + \alpha R_{ui}$ is introduced to assign appropriate weights to observed and unobserved interactions.¹⁷

The closed-form mathematical solution to update the latent vector for a specific user u , while holding the entire item matrix Y constant, is elegantly formulated as:

$$x_u = (Y^T C^u Y + \lambda I)^{-1} Y^T C^u p(u)$$

Where the components are defined as follows:

- $Y^T C^u Y$ represents the weighted covariance matrix of the items.
- λI constitutes the L_2 regularization term, crucial for preventing catastrophic overfitting.
- $p(u)$ is the binarized preference vector (defined as 1 if $R_{ui} > 0$, else 0).

From a pure-function architectural perspective, implementing the entire iterative orchestration loop within a single function violates the isolation principle. Optimization loops inherently dictate convergence criteria, extensive logging, and mutable state management. Therefore, the canonical atomic unit is the single vector update step itself.¹⁸ In pure NumPy, utilizing `numpy.linalg.solve` to determine the update rather than explicitly computing the matrix inversion via `numpy.linalg.inv` is universally recommended. Explicit inversion of nearly singular matrices frequently introduces severe floating-point inaccuracies and computational bottlenecks, whereas linear solvers offer enhanced numerical stability.

Furthermore, a significant algebraic optimization exists for implicit feedback ALS. The computation of $Y^T C^u Y$ for every single user is prohibitively slow because the confidence matrix C^u varies per user. By algebraically rewriting the term as $Y^T Y + Y^T (C^u - I) Y$, the global term $Y^T Y$ can be precomputed once per iteration. The $(C^u - I)$ matrix contains non-zero elements *only* where the user has actually interacted with an item, allowing the computation to scale with the number of non-zero interactions rather than the total catalog size. A highly optimized pure function must leverage this mathematical property.

Name: `als_user_update`

Description: Computes the closed-form Alternating Least Squares (ALS) update for a single user's latent factor vector, leveraging precomputed covariance and interaction sparsity.

Source: Hu, Koren, Volinsky (2008) "Collaborative Filtering for Implicit Feedback Datasets"; implicit library (MIT) conceptual translation.

License: MIT

Concept type: optimization

Signature: `(user_interactions: scipy.sparse.csr_matrix, item_factors: NDArray, yt_y: NDArray, regularization: float, alpha: float = 40.0) -> NDArray`

Pure function boundary: Purely mathematical linear solver. Ingests a sparse vector of interactions for one user, the dense global item factors, precomputed $Y^T Y$ covariance, and scalar hyperparameters. Returns the updated dense latent vector for that specific user.

Contracts:

- require: `regularization > 0.0`

- require: `user_interactions.shape == item_factors.shape`
- require: `yt_y.shape == (item_factors.shape, item_factors.shape)`
- ensure: `output.shape == (item_factors.shape,)`
- ensure: output contains no NaN or Inf values.

Witness: Provide an identity matrix for `item_factors` and a single isolated interaction. Verify the output vector accurately aligns with the specific interaction index, correctly scaled by the regularized confidence scalar.

Dependencies: `numpy`, `scipy.sparse`, `numpy.linalg.solve`

CDG stages covered: `h_and_m/als_matrix_factorization`, `general/implicit_cf`

Resolving the Cold-Start Problem within Pure Boundaries

In the domain of recommender systems, the term "cold-start" refers to the system's fundamental inability to infer properties of new users or items due to an absolute absence of interaction history.²¹ Within the strict architecture of pure functions, cold-start handling cannot be abstracted away; it must be explicitly encoded as a precondition routing protocol.

For instance, the `als_user_update` mathematical formulation cannot proceed if the user interaction matrix is entirely populated by zeros. In such a scenario, the user-specific covariance matrix evaluates to zero, and the subsequent linear solving operation depends entirely on the static regularization parameter, ultimately producing a meaningless, uniform vector. The upstream pipeline orchestration must proactively trap cold-start entities before they cross the pure-function boundary. Typically, this involves substituting global mean latent vectors or invoking entirely separate content-based functions.²² The pure function's contract enforces this logic strictly: an entity must possess a minimum of one valid interaction to successfully proceed through the collaborative filtering primitives.

Data Assembly and Ranking Features

Following the candidate generation phases, the retrieved candidates must be accurately scored and meticulously ranked. In modern two-stage systems—such as those employing Gradient Boosted Decision Trees (GBDT) like LightGBM, XGBoost, or CatBoost—the sophistication of the feature engineering directly dictates the predictive precision of the ranker.²³

4. Exponential Time Decay for Item Popularity

Recommending items purely based on their absolute, all-time historical popularity heavily favors older, established items, completely obscuring rapidly trending products. This is a fatal conceptual flaw in highly dynamic datasets like the fast-fashion H&M catalog, where item lifecycles are brief and seasonal relevance is paramount.²⁵ The standard mathematical remedy to this problem is modeling popularity not as a static count, but with an exponential decay curve.

The popularity score S_i for a specific item i decays continuously over time. Rather than simply counting raw occurrences within a static rolling window, the function dynamically aggregates time-weighted occurrences.²⁷

$$S_i = \sum_{e \in E_i} e^{-\lambda(t_{now} - t_e)}$$

Where E_i represents the complete set of historical interaction events for item i , t_{now} is the current reference time for the system state, t_e is the absolute timestamp of the individual event, and the decay constant λ is directly parameterized by a highly configurable half-life variable ($t_{1/2}$)²⁹:

$$\lambda = \frac{\ln(2)}{t_{1/2}}$$

Vectorizing this operation in NumPy requires sophisticated array manipulation. The function must compute the discrete time deltas as a dense array, apply the continuous exponential function to the entire array simultaneously via underlying C-optimized routines, and then utilize numpy.bincount or grouped aggregations to accurately sum the decayed weights mapped to each specific item ID. This methodology entirely avoids slow, iterative Python for loops and ensures high-throughput feature generation.³⁰

Name: item_popularity_decay

Description: Calculates the time-decayed popularity score for an entire catalog of items, heavily weighting recent interactions using a smooth exponential half-life function.

Source: General mathematical modeling in predictive scoring and standard Kaggle temporal feature engineering (e.g., H&M competition).

License: MIT

Concept type: data_assembly

Signature: (item_ids: NDArray, timestamps: NDArray, reference_time: float, half_life: float) -> NDArray

Pure function boundary: Receives dense arrays of historical events and scalar time boundaries. Calculates array-wide decay weights and aggregates them. Returns a 1D array where the index explicitly corresponds to the item ID and the value is the decayed popularity score.

Contracts:

- require: len(item_ids) == len(timestamps)
- require: half_life > 0.0

- require: `reference_time >= max(timestamps)`
- ensure: all output values are strictly non-negative.
- ensure: older timestamps mathematically result in exponentially smaller score contributions.

Witness: Pass two interactions for the exact same item, one at `reference_time` and one exactly one `half_life` earlier. The resulting combined score must exactly equal 1.5.

Dependencies: `numpy`

CDG stages covered: `h_and_m/popularity_scoring`, `otto/trending_items`

5. Session-Based Contextual Features

User behavior within an active, ongoing session provides the single most predictive signal for their immediate next action. Extracting precise session context—such as user dwell time, raw interaction frequency, and categorical behavioral shifts—is crucial for the ranking algorithm.⁹

A pure function responsible for session characteristic extraction must evaluate an isolated sequence of timestamps and corresponding item IDs. Statistical aggregations include the total observed session duration ($\max(T) - \min(T)$), the absolute number of unique items explored (calculated via `len(numpy.unique(items))`), and the average chronological gap between discrete interactions (`numpy.mean(numpy.diff(T))`). By wrapping these disparate aggregations into a single, cohesive dictionary-emitting pure function, the feature engineering logic remains highly portable and easily parallelizable across Dask partitions.

Name: `session_features`

Description: Extracts critical aggregate statistical features from a chronological sequence of active session interactions.

Source: Analytical insights drawn from winning solutions of the OTTO Multi-Objective Recommender System competition.

License: MIT

Concept type: `data_assembly`

Signature: `(item_sequence: list[int], timestamps: NDArray) -> dict[str, float]`

Pure function boundary: Ingests a single session's raw arrays and outputs a flat dictionary of scalar floats, perfectly structured for immediate ingestion into tabular models (e.g., LightGBM).

Contracts:

- require: `len(item_sequence) == len(timestamps)`
- require: timestamps are strictly monotonically increasing.
- ensure: dictionary contains keys `'session_length'`, `'unique_items'`, `'avg_gap'`, `'duration'`.
- ensure: if `len(timestamps) == 1`, `'avg_gap'` and `'duration'` safely default to 0.0.

Witness: Provide a sequence of 3 identical items systematically spaced 10 seconds apart.

Assert `'unique_items'` evaluates to 1 and `'avg_gap'` evaluates to 10.0.

Dependencies: `numpy`

CDG stages covered: otto/session_feature_engineering

6. User-Item Interaction Affinity

While broad global popularity and immediate session context are exceptionally valuable, highly personalized recommendation relies fundamentally on the specific historical affinity between a user and a candidate item. Feature engineering in this vector revolves heavily around deep historical engagement: has the user purchased this specific item before? How many discrete times? Exactly how many days have elapsed since the final interaction?⁷

Affinity aggregations calculate discrete statistical scalars for the explicit tuple (u, i) . Given a complete history tensor of events E_u for a user, rigorously filtered for the presence of item i , the system computes:

1. **Total Interaction Count:** $|E_{u,i}|$
2. **Interaction Recency:** $t_{now} - \max(t \in E_{u,i})$

If the candidate item has never been historically interacted with, these pure functions must deterministically handle the empty array inputs. They typically return predefined fallback constants (e.g., 0 for interaction counts, and a defined maximum boundary or -1.0 for recency) to prevent generating NaN values that would immediately crash tree-based rankers during inference.

Name: user_item_affinity

Description: Computes highly personalized affinity features representing the deep historical relationship between a specific user and a candidate item.

Source: H&M Personalized Fashion Recommendations top-tier feature engineering practices for GBDT rankers.

License: MIT

Concept type: data_assembly

Signature: (user_item_timestamps: NDArray, reference_time: float) -> dict[str, float]

Pure function boundary: Operates strictly on the pre-filtered array of interaction timestamps for a known user-item pair. Does not perform the expensive matrix filtering itself, maintaining $O(1)$ complexity within the pure bound. Returns a clean dictionary of scalars.

Contracts:

- require: reference_time $\geq \max(\text{user_item_timestamps}, \text{default}=0)$
- ensure: 'interaction_count' ≥ 0
- ensure: if user_item_timestamps is entirely empty, 'recency' reliably returns -1.0 or a

designated NaN equivalent.

Witness: Pass an array of 5 dense timestamps. Assert 'interaction_count' evaluates to 5 and 'recency' strictly reflects the delta between reference_time and the maximum observed timestamp.

Dependencies: numpy

CDG stages covered: h_and_m/user_item_features, instacart/repurchase_probability

Advanced Training Strategies: Negative Sampling

While candidate generation and feature extraction define the inference pipeline, training the underlying recommendation models—particularly the neural collaborative filtering or GBDT ranking components—requires a carefully constructed dataset of both positive interactions and negative counter-examples.⁷ Pure functions play a critical role in orchestrating these sampling strategies deterministically.

Negative sampling addresses the implicit feedback problem: while a user clicking an item is a strong positive signal, the absence of a click does not inherently guarantee a negative preference; the user may simply not have encountered the item.³³ Effective training requires constructing a balanced proxy of negative signals.

The industry employs three primary sampling strategies, each with distinct mathematical properties that dictate how a pure function must behave:

Sampling Strategy	Mechanism	Recommender System Application
Uniform Random Sampling	Selects unobserved items with equal probability across the entire catalog.	Establishes a basic baseline. Prone to generating "easy" negatives that the model trivially dismisses.
Popularity-Biased Sampling	Selects unobserved items proportionally to their global frequency in the dataset.	Highly effective for mitigating popularity bias. Forces the model to learn personalized signals rather than simply recommending best-sellers. ³⁴
Hard Negative Mining	Selects items that the candidate generation stage scored highly, but the user	Crucial for the final GBDT ranking stage. Trains the model to discern nuanced differences between

	ultimately ignored.	closely related items. ³⁵
--	---------------------	--------------------------------------

A pure function designed for negative sampling must rely heavily on a provided random seed or an explicitly passed pseudo-random number generator (PRNG) state to ensure that execution remains completely deterministic and reproducible across distributed training clusters. The function receives the array of observed positive item IDs, the array of potential candidate item IDs (or sampling probabilities), the desired sample count, and the PRNG state, returning an array of selected negative indices.

Ensemble and Fusion Mechanisms

Modern recommendation pipelines rarely rely exclusively on a single monolithic retrieval algorithm. Instead, they employ robust ensembles—combining disparate candidate streams sourced from collaborative filtering matrices, deep content-based vector searches, and rigid popularity-based heuristics.³² Merging these highly heterogeneous candidate lists poses a significant mathematical challenge, as the similarity scores generated from different algorithms inhabit entirely disparate statistical scales (e.g., Euclidean distances vs. cosine similarities vs. raw count aggregations).³⁷

7. Reciprocal Rank Fusion (RRF)

Reciprocal Rank Fusion, originally formalized by Cormack, Clarke, and Büttcher in 2009, elegantly circumvents complex score normalization issues by entirely discarding the raw algorithmic scores.³⁸ Instead, the RRF method fuses multiple lists based exclusively on the relative rank positions of the items within each respective list.⁴⁰

The underlying mathematical formulation is notably straightforward yet remarkably effective.

Given a set of ranked candidate lists L , the aggregated RRF score for a specific candidate document or item d is defined as the mathematical sum of its reciprocal ranks across all individual lists in which it appears:

$$RRF_Score(d) = \sum_{l \in L} \frac{1}{k + rank_l(d)}$$

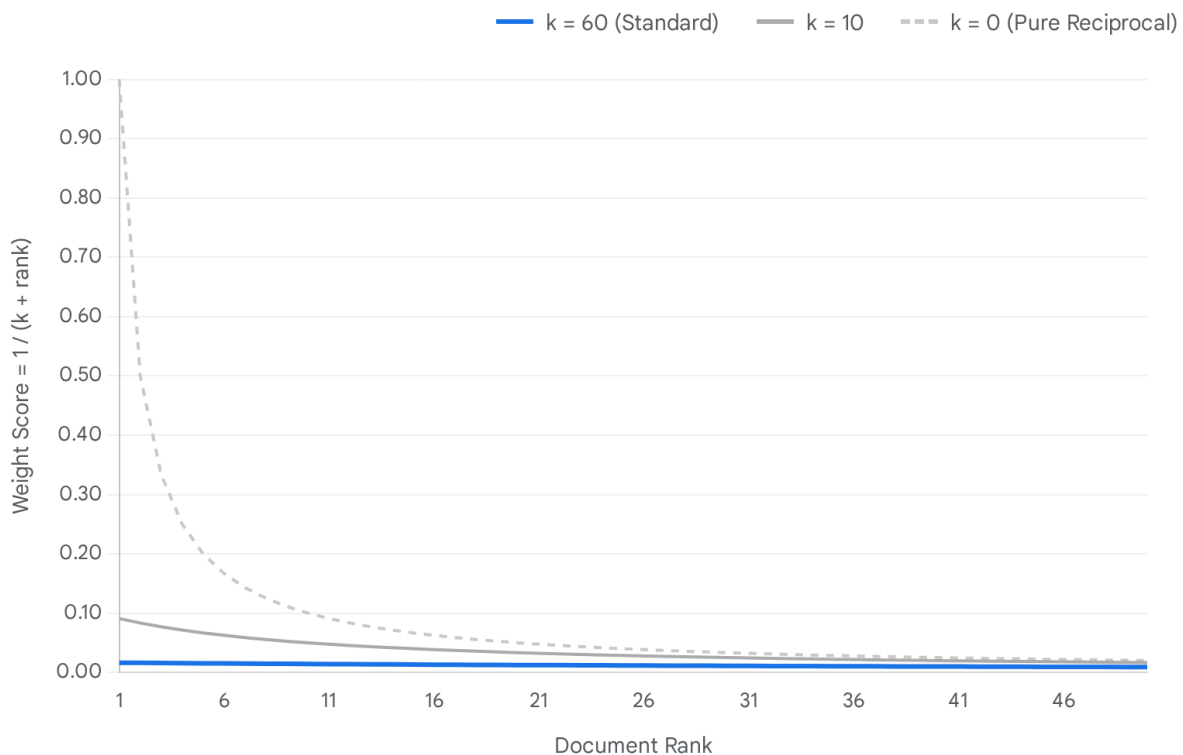
Where $rank_l(d)$ represents the 1-based rank position of item d within list l .

The Hyperparameter k

The smoothing constant k is critical for calibrating the fusion process. Empirical validation by Cormack et al. established $k = 60$ as a highly robust default value.⁴¹ The presence of the k parameter effectively flattens the extreme mathematical convexity of the standard $1/x$ curve.

Without the inclusion of k , an item randomly ranked #1 in a highly flawed algorithm and #100 in an excellent algorithm might unfairly dominate the fused output list. By strictly setting $k = 60$, the algorithmic influence intentionally shifts away from rewarding absolute top-rank dominance toward rewarding consistent, high-tier placement across multiple independent strategies.³⁹

Impact of Parameter 'k' on Reciprocal Rank Scoring



By setting $k=60$, the algorithm flattens the severe exponential drop-off seen in lower k values, ensuring that items consistently ranked in the top 20 across multiple systems outscore an item ranked #1 in a single, potentially anomalous system.

Data sources: [Deval Shah \(Medium\)](#), [Cormack et al., 2009](#)

The algorithmic complexity of a pure Python or NumPy implementation of RRF requires iterating systematically through the provided lists, maintaining a hash map or an aggregated index array of scores. This results in an $O(N)$ time complexity, where N is the absolute total number of candidate occurrences evaluated across all lists. By returning a sorted subset, the

function efficiently completes the fusion process.

Name: `reciprocal_rank_fusion`

Description: Fuses multiple independent ranked lists of candidate items into a single unified ranking using the Reciprocal Rank Fusion algorithm.

Source: Cormack, G. V., Clarke, C. L., & Buettcher, S. (2009). "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods." SIGIR '09.

License: MIT

Concept type: searching

Signature: `(ranked_lists: list[list[int]], k: int = 60, top_n: int = 100) -> list[int]`

Pure function boundary: Ingests a multi-dimensional list of integer lists (representing item IDs in tightly ranked order) and configuration scalars. Returns a single flat list of integer item IDs sorted mathematically by their aggregated RRF score. No external validation of item IDs occurs.

Contracts:

- require: `k > 0`
- require: `top_n > 0`
- ensure: output list contains strictly unique integers.
- ensure: `len(output) <= top_n`

Witness: Provide list A: and list B: . Verify that item 3 and item 2 successfully receive aggregated mathematical scores from both lists and reliably outrank items appearing in only a single list.

Dependencies: `collections.defaultdict` (standard library)

CDG stages covered: `general/candidate_ensembling`, `h_and_m/hybrid_search`

By adhering rigorously to these stringent functional contracts and mathematical boundaries, machine learning engineering teams can construct recommender pipelines that are highly scalable, fundamentally testable, and robust against the silent state-corruption errors that consistently plague older, monolithic architectures.

Works cited

1. otto-de/recsys-dataset: A real-world e-commerce dataset for session-based recommender systems research. - GitHub, accessed April 30, 2026, <https://github.com/otto-de/recsys-dataset>
2. aldensiol/FashoReco: A repository for a H&M Recommendation System. Done for BT4222 - GitHub, accessed April 30, 2026, <https://github.com/aldensiol/FashoReco>
3. Functional Programming in Python - Ada Beat, accessed April 30, 2026, <https://adabeat.com/fp/functional-programming-in-python/>
4. Functional Programming: "Different Approach to Problem Solving by Pure

- Functions” in Python | by Ayesha Javed | Medium, accessed April 30, 2026,
<https://medium.com/@ayshajavd/functional-programming-different-approach-to-problem-solving-by-pure-functions-in-python-5feaa514a576>
5. NumPy User Guide, accessed April 30, 2026,
<https://numpy.org/doc/2.1/numpy-user.pdf>
 6. Build Efficient Recommender Systems with Co-Visitation Matrices and RAPIDS cuDF, accessed April 30, 2026,
<https://developer.nvidia.com/blog/build-efficient-recommender-systems-with-co-visitation-matrices-and-rapids-cudf/>
 7. Silver Medal Solution on Kaggle H&M Personalized Fashion Recommendations, accessed April 30, 2026,
<https://ajisamudra.medium.com/silver-medal-solution-on-kaggle-h-m-personalized-fashion-recommendations-a0878e1eae63>
 8. Building a Recommender System using Machine Learning - Leonie Monigatti, accessed April 30, 2026,
<https://www.leonimonigatti.com/blog/recommender-system-ml.html>
 9. GitHub - Fedorov-Artem/kaggle_OTTO: code for kaggle OTTO competition (got silver medal), accessed April 30, 2026,
https://github.com/Fedorov-Artem/kaggle_OTTO
 10. Bronze Medal Solution on Kaggle OTTO — Multi-Objective Recommender System, accessed April 30, 2026,
<https://ajisamudra.medium.com/bronze-medal-solution-on-kaggle-otto-multi-objective-recommender-system-d0720aed4712>
 11. Multi-dimensional Sparse Arrays in SciPy - Quansight Labs, accessed April 30, 2026, <https://labs.quansight.org/blog/multi-dimensional-sparse-arrays>
 12. How to convert co-occurrence matrix to sparse matrix - Stack Overflow, accessed April 30, 2026,
<https://stackoverflow.com/questions/15030047/how-to-convert-co-occurrence-matrix-to-sparse-matrix>
 13. Implementing the Alternating Least Squares Algorithm | CodeSignal Learn, accessed April 30, 2026,
<https://codesignal.com/learn/courses/diving-deep-into-collaborative-filtering-techniques-with-als/lessons/implementing-the-alternating-least-squares-algorithm>
 14. Nonnegative Matrix Factorization: The Alternating Least Squares Method - Stack Overflow, accessed April 30, 2026,
<https://stackoverflow.com/questions/32612502/nonnegative-matrix-factorization-the-alternating-least-squares-method>
 15. dnyanshwalwadkar/H-M-Personalized-Fashion-Recommendations - GitHub, accessed April 30, 2026,
<https://github.com/dnyanshwalwadkar/H-M-Personalized-Fashion-Recommendations>
 16. Matrix Factorization with Alternating Least Squares - Akshay Kulkarni, accessed April 30, 2026, <https://akshay-a-kulkarni.github.io/reports/ALS.pdf>
 17. Explicit Matrix Factorization: ALS, SGD, and All That Jazz | Ethan Rosenthal, accessed April 30, 2026,

- <https://www.ethanrosenthal.com/2016/01/09/explicit-matrix-factorization-sgd-als/>
18. Alternating Least Squares - Recommendation-Systems Guide Book, accessed April 30, 2026, <https://reco.mlguidebook.com/en/latest/notebooks/als.html>
 19. 1_ALSWR, accessed April 30, 2026, https://ethen8181.github.io/machine-learning/recsys/1_ALSWR.html?utm_source
 20. Reco Guide Book, accessed April 30, 2026, https://reco.mlguidebook.com/_downloads/en/latest/pdf/
 21. Study of collaborative filtering recommendation with user clustering incorporating implicit social relationships and trust relationships - PMC, accessed April 30, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12500082/>
 22. MAGUIRAT/HMRecommenderSystem: LTR-based Recommender System for H&M Personalized Fashion Recommendations Kaggle Competition - GitHub, accessed April 30, 2026, <https://github.com/MAGUIRAT/HMRecommenderSystem>
 23. nicolaivicol/otto-recommender: Solution to kaggle competition OTTO – Multi-Objective Recommender System: <https://www.kaggle.com/competitions/otto-recommender-system> · GitHub - GitHub, accessed April 30, 2026, <https://github.com/nicolaivicol/otto-recommender>
 24. OTTO – Multi-Objective Recommender System - Kaggle, accessed April 30, 2026, <https://www.kaggle.com/c/otto-recommender-system/discussion/372976>
 25. Blog #7: How to use our AI Simulated H&M Store Front, accessed April 30, 2026, <https://www.sempoa.ai/blog/007>
 26. Shanmukhi1920/H-M-Recommendation-System: Develop personalized product recommendations for H&M's users using transaction history, customer data, and product metadata (including text descriptions) to enhance shopping experience and sustainability. - GitHub, accessed April 30, 2026, <https://github.com/Shanmukhi1920/H-M-Recommendation-System>
 27. Counters in Recommendations: From Exponential Decay to Position Debiasing | by Michael Roizner, accessed April 30, 2026, <https://roizner.medium.com/counters-in-recommendations-from-exponential-decay-to-position-debiasing-30a6175bba5>
 28. Using Time Decay in Predictive Lead Scoring | by Filip Vozarevic | Medium, accessed April 30, 2026, <https://medium.com/@filip.vozarevic/using-time-decay-in-predictive-lead-scoring-852de2052ea>
 29. Creating Recommendations for User-Created Content in the Vocolize Application - Diva-portal.org, accessed April 30, 2026, <https://www.diva-portal.org/smash/get/diva2:1648846/FULLTEXT01.pdf>
 30. create a list of exponential decay with a predetermine half-life python - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/43613407/create-a-list-of-exponential-decay-with-a-predetermine-half-life-python>
 31. nlztrk/OTTO-Multi-Objective-Recommender-System - GitHub, accessed April 30, 2026, <https://github.com/nlztrk/OTTO-Multi-Objective-Recommender-System>

32. Building a Two-Stage Fashion Recommendation System with Recall and Learning-to-Rank | by Anila Chang | Medium, accessed April 30, 2026, <https://medium.com/@anilaChang/building-a-two-stage-fashion-recommendation-system-with-recall-and-learning-to-rank-3397d7220ced>
33. Explaining Negative Sampling in Recommender Systems | BCG X, accessed April 30, 2026, <https://www.bcg.com/x/the-multiplier/explaining-negative-sampling-in-recommender-systems>
34. Implement a RecSys, Chapter 3: Negative Sampling - DvQ, accessed April 30, 2026, <https://dvquys.com/projects/implement-recsys/c3/>
35. Negative Sampling in Recommendation: A Survey and Future Directions - arXiv, accessed April 30, 2026, <https://arxiv.org/html/2409.07237v1>
36. tuansunday05/ArticleRecommenderSystem: Using hybrid recommender system with apriori algorithm, content-based and collaborative filtering method for predicting users interactions and then recommend them for users. - GitHub, accessed April 30, 2026, <https://github.com/tuansunday05/ArticleRecommenderSystem>
37. Advanced RAG — Understanding Reciprocal Rank Fusion in Hybrid Search, accessed April 30, 2026, <https://glaforge.dev/posts/2026/02/10/advanced-rag-understanding-reciprocal-rank-fusion-in-hybrid-search/>
38. Implementing Reciprocal Rank Fusion (RRF) in Python - Krystian Safjan's Blog, accessed April 30, 2026, <https://safjan.com/implementing-rank-fusion-in-python/>
39. Reciprocal Rank Fusion (RRF) explained in 4 mins — How to score results from multiple retrieval methods in RAG | by Deval Shah | Medium, accessed April 30, 2026, <https://medium.com/@devalshah1619/mathematical-intuition-behind-reciprocal-rank-fusion-rrf-explained-in-2-mins-002df0cc5e2a>
40. Hybrid Search in RAG: Dense + Sparse (BM25/SPLADE), Reciprocal Rank Fusion, and When to Use Which! | by Vaibhav Dixit | Mar, 2026 | GoPenAI, accessed April 30, 2026, <https://blog.gopenai.com/hybrid-search-in-rag-dense-sparse-bm25-splade-reciprocal-rank-fusion-and-when-to-use-which-fafe4fd6156e>
41. Reciprocal Rank Fusion outperforms Condorcet and individual Rank Learning Methods - University of Waterloo, accessed April 30, 2026, <https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf>
42. How To Implement Contextual RAG From Anthropic - Together AI Docs, accessed April 30, 2026, <https://docs.together.ai/docs/how-to-implement-contextual-rag-from-anthropic>